Analyzing the Role of AI Assisted Programming in Modern Software Development

A Case Study Using a Java Library Management System

Module: Advanced Programming Concepts
Assessment: AI Assisted Programming
Student ID: EIT | S17229
Department of Physics, University of Colombo
IT 3003 - Advanced Programming Techniques

Table of Contents

Task 1: Conceptual Understanding

1.1 What is AI Assisted Programming?

AI-assisted programming refers to the use of artificial intelligence tools, typically built on large language models, to support software developers throughout the coding process. Rather than replacing the programmer, these tools act as an intelligent assistant that can generate code from natural-language descriptions, explain existing code, suggest fixes for bugs, refactor implementations, and produce supporting artifacts such as documentation and diagrams. In this assignment, tools such as GitHub Copilot-style code generation and conversational assistants were used to help translate, restructure, and improve a Java program while the developer retained control over design decisions and final acceptance of the generated code.

1.2 Types of AI Coding Tools

- Code generation tools produce new code, functions, or entire modules from natural language prompts (e.g., GitHub Copilot, ChatGPT, Claude).
- Debugging assistants analyze stack traces and logic errors and propose corrected code or explanations of the fault.
- Refactoring and code-quality tools suggest structural improvements such as converting classes to records, replacing loops with Streams, or introducing custom exceptions.
- Documentation and diagram generators automatically produce README files, inline comments, and UML diagrams (such as the PlantUML diagrams used in this assignment).
- Code review and static-analysis assistants flag potential security issues, code smells, or style violations before code is merged.

1.3 Advantages in Software Development

- Faster development boilerplate code, data classes, and repetitive menu logic can be generated in seconds rather than written by hand.
- Improved code quality AI tools often suggest modern language features (Java records, Streams API) and defensive patterns such as validation and custom exceptions.
- Better documentation AI can generate consistent comments, README files, and UML diagrams that developers might otherwise skip under time pressure.

- Learning support less experienced developers can see idiomatic examples of a pattern and learn by comparison, as demonstrated by comparing Version A and Version B of this project.
- Lower barrier to entry for new APIs developers can adopt unfamiliar APIs (such as java.nio.file) more quickly with AI guidance.

1.4 Risks and Limitations

- Hallucinated code AI tools can generate code that compiles but uses non-existent methods, incorrect library behaviour, or subtly wrong logic that must be manually verified.
- Over-dependency excessive reliance on AI-generated code can weaken a developer's own problem-solving and debugging skills over time.
- Security issues AI suggested code may omit proper input validation, use insecure file handling, or introduce injection-style vulnerabilities if not reviewed carefully.
- Lack of context awareness AI tools may not fully understand the wider system architecture, leading to suggestions that conflict with existing design decisions.
- Licensing and originality concerns generated code may closely resemble licensed training examples, raising intellectual-property questions in professional settings.
- False confidence well formatted, professional-looking output can appear more trustworthy than it actually is, making review discipline essential.

Task 2: Practical Implementation

2.1 Problem Selection

The selected problem is a Library Management System of moderate complexity. The system allows a librarian to add books to the catalogue, register members, allow members to borrow and return books, list all books and members, and (in the improved version) search the catalogue by ID, title, or author. This problem was chosen because it involves realistic entity relationships (books and members), basic business rules (a book cannot be borrowed twice, a member cannot return a book they did not borrow), and enough scope to meaningfully demonstrate the difference between a traditional implementation and an AI assisted one.

Two versions of the solution were developed in Java. Version A was written using a traditional, beginner-friendly object oriented approach with no AI assistance. Version B was developed with AI assistance and rebuilt using modern Java features, including records, the Streams API, custom exceptions, and file-based persistence.

2.2 Version A – Traditional Programming Approach

Version A uses three plain Java classes: Book, Member, and Library. Data is stored in ArrayList collections that only exist in memory for the duration of the program run, so nothing is preserved after the program exits. Validation and error handling are implemented by returning plain message strings from each method (e.g., "Book not found."), and the console menu drives all interaction through a Scanner and a chain of if / else if statements.

Key excerpt Library.borrowBook() (Version A)

```
public String borrowBook(String memberId, String bookId) {
    Member member = findMember(memberId);
    Book book = findBook(bookId);

    if (member == null) {
        return "Member not found.";
    }
    if (book == null) {
        return "Book not found.";
    }
    if (!book.isAvailable()) {
        return "Book is already borrowed.";
    }

    book.setAvailable(available: false);
    member.getBorrowedBooks().add(bookId);
    return "Book borrowed successfully.";
}
```

This method demonstrates the traditional style: it performs a linear search for the member and book, checks preconditions with simple if statements, and returns a human readable status string rather than throwing an exception.

Sample console session | Version A

```
Library Management System - Version A
1. Add book
2. Add member
3. Borrow book
4. Return book
5. List books
6. List members
0. Exit
Select option: 1
Book ID: PH 3006
Title: Advance Analogue and Digital Electronics
Author: Prof. Lakmal Weerawarne
Book added successfully.
```

```
Library Management System - Version A
1. Add book
2. Add member
3. Borrow book
4. Return book
5. List books
6. List members
0. Exit
Select option: 2
Member ID: s17229
Name: Kaveesha Induwara
Member added successfully.
```

2.3 Version B AI Assisted Programming Approach

Version B was restructured with AI assistance into a service-based design. Book and Member are immutable Java records, updates are handled by producing new record instances (withAvailability,with Borrowed Books), and all business logic is centralized in a LibraryService class. A custom LibraryException replaces string-based error messages, input is validated through a required() helper, and every state change is persisted to library_data.txt using standard java.nio.file I/O so that data survives between runs. A Streams-based searchBooks() method was also added, which has no equivalent in Version A.

Key excerpt – LibraryService.borrowBook() (Version B)

```java
public void borrowBook(String memberId, String bookId) throws LibraryException {
    Member member = getMember(memberId);
    Book book = getBook(bookId);

    if (!book.available()) {
        throw new LibraryException(book.title() + " is already borrowed.");
    }
    if (member.borrowedBooks().contains(book.bookId())) {
        throw new LibraryException(message: "The selected member already has this book.");
    }

    List<String> updatedBorrowedBooks = new ArrayList<>(member.borrowedBooks());
    updatedBorrowedBooks.add(book.bookId());
    members.put(member.memberId(), member.withBorrowedBooks(updatedBorrowedBooks));
    books.put(book.bookId(), book.withAvailability(newAvailability: false));
    save();
}
```

Key excerpt – Streams-based search (AI suggested, Version B)

```java
public List<Book> searchBooks(String keyword) throws LibraryException {
    String loweredKeyword = required(keyword, fieldName: "Search keyword").toLowerCase();
    return books.values()
            .stream()
            .filter(book -> book.bookId().toLowerCase().contains(loweredKeyword)
                    || book.title().toLowerCase().contains(loweredKeyword)
                    || book.author().toLowerCase().contains(loweredKeyword))
            .sorted(Comparator.comparing(Book::bookId))
            .collect(Collectors.toList());
}
```

The comment beginning "// AI Suggestion:" above the method (see full source code in the appendix) marks this as an AI-assisted addition, as required by the assignment instructions.

Sample console session – Version B

```
Library Management System - Version B
1. Add book
2. Add member
3. Borrow book
4. Return book
5. List books
6. List members
0. Exit
Select option: 5
B001 | Clean Code | Robert C. Martin | Available
B002 | Python Crash Course | Eric Matthes | Available
```

2.4 UML Diagrams

The following diagrams describe the design of the Library Management System (based on the Version B design, which reflects the final class structure).

2.4.1 Use Case Diagram

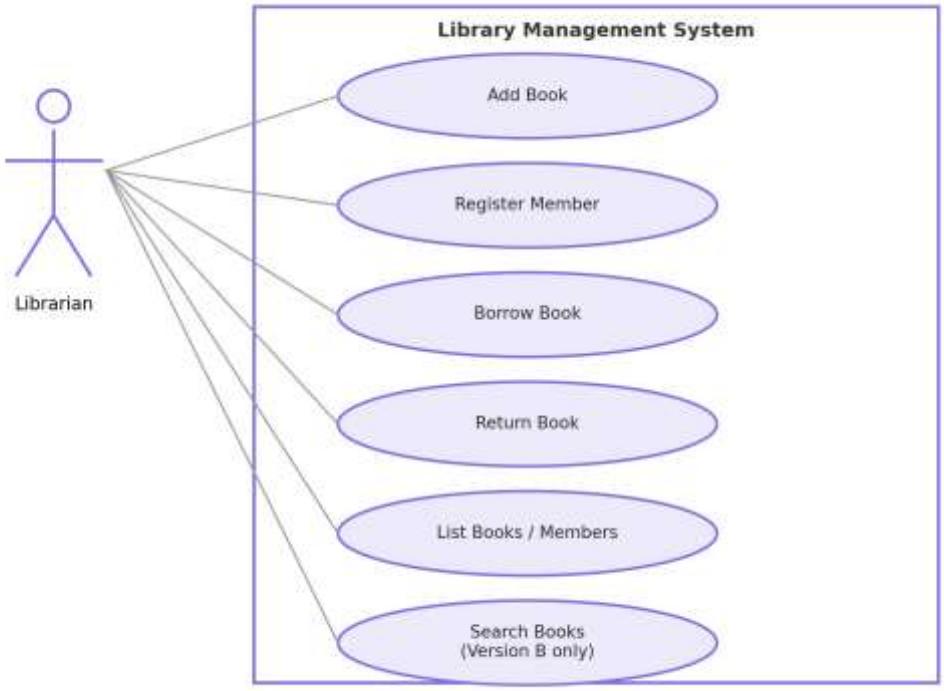

Figure 1: Use Case Diagram – Library Management System

2.4.2 Class Diagram

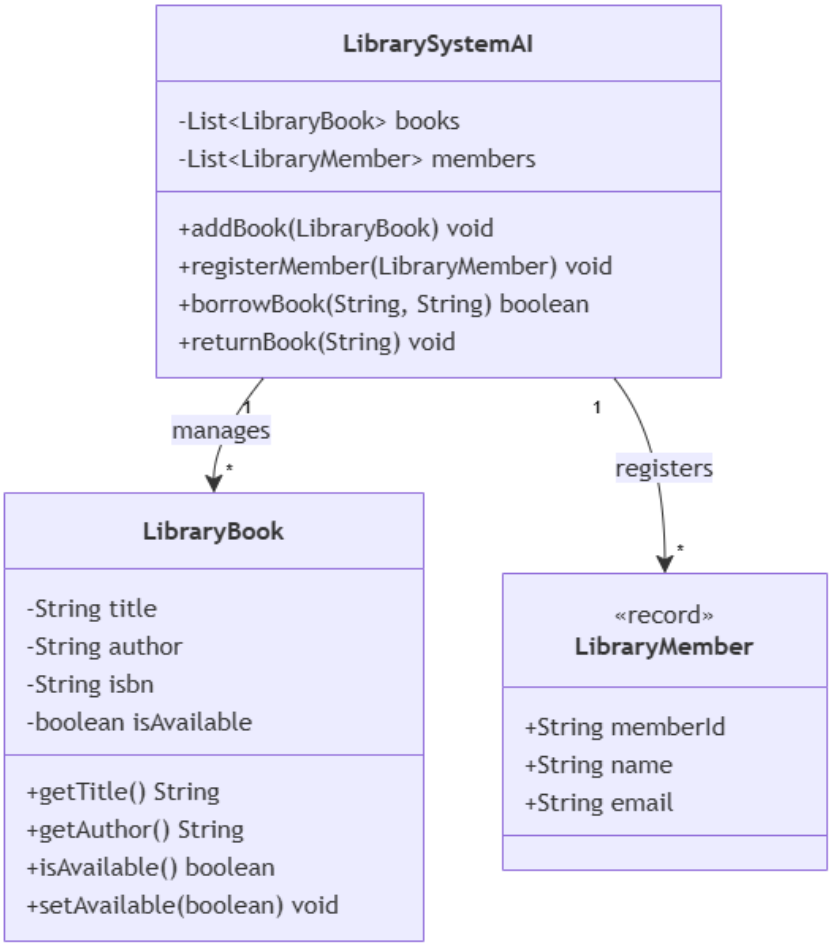

Figure 2: Class Diagram – Library Management System

2.4.3 Activity Diagram

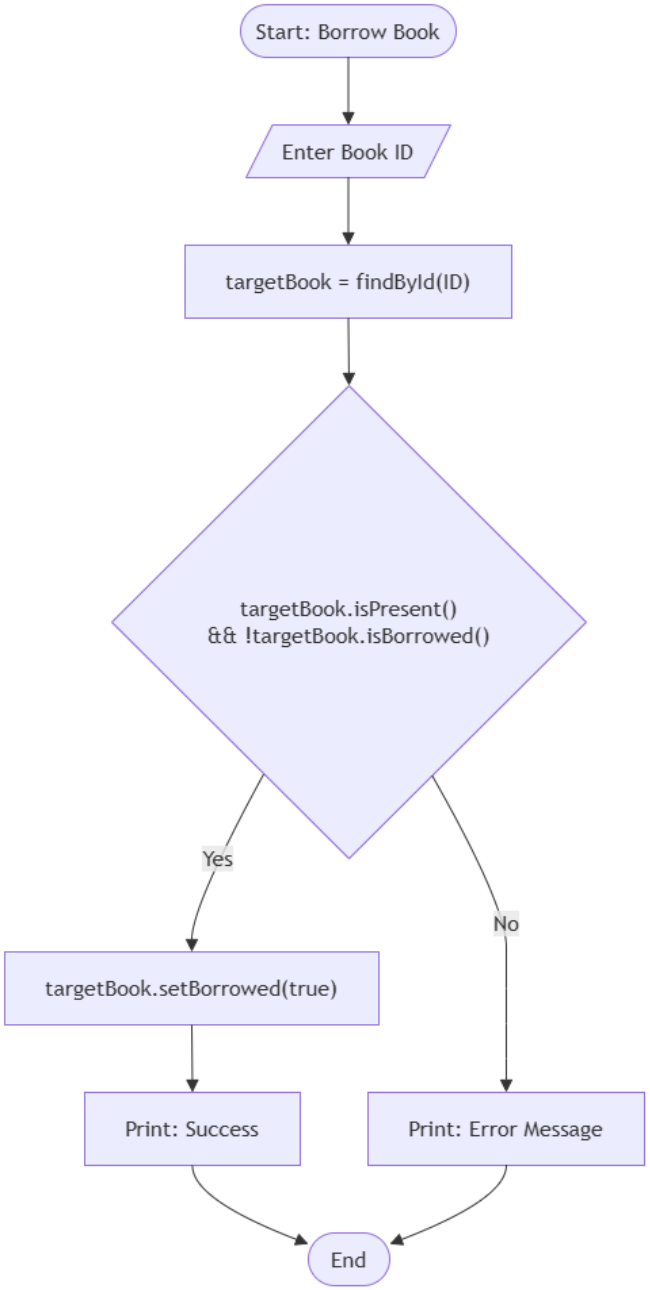

Figure 3: Activity Diagram – Borrow Book Process

2.4.4 Sequence Diagram (Optional)

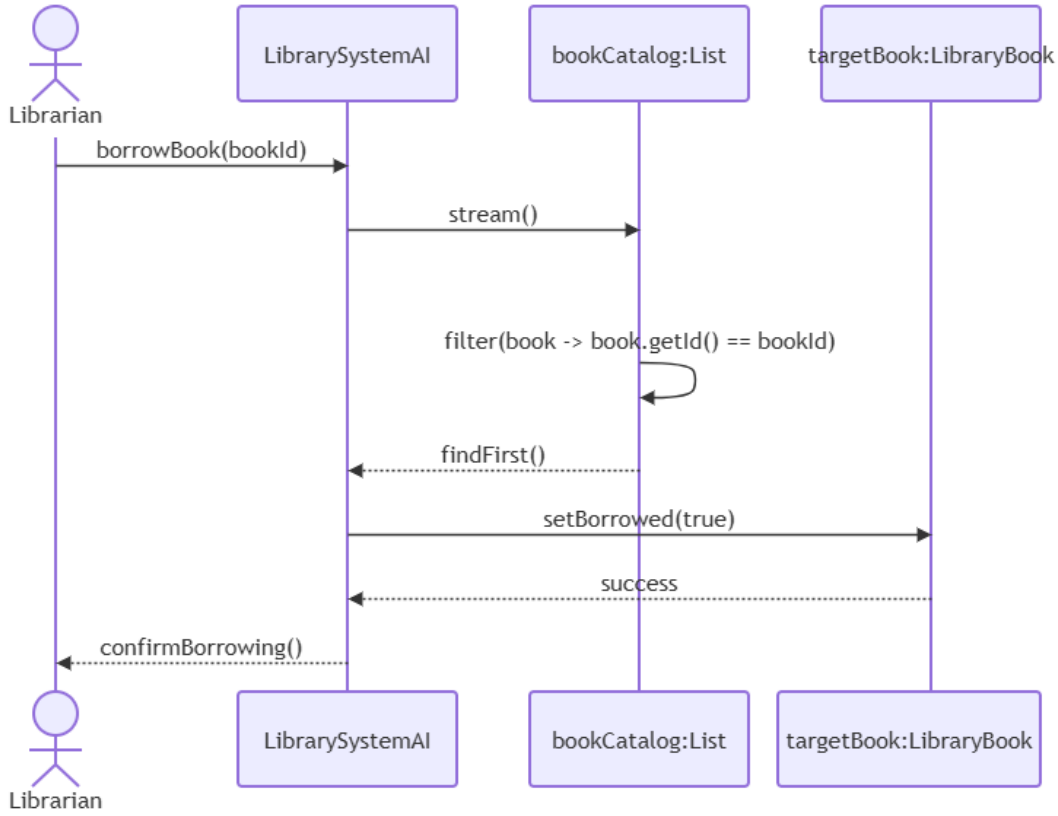

Figure 4: Sequence Diagram Borrow Book Flow

2.5 Comparison of Version A and Version B

Area	Version A (Traditional)	Version B (AI Assisted)
Data model	Plain Java classes with getters/setters	Immutable Java records
Storage	In-memory only (lost on exit)	File persistence (library_data.txt)
Collections	ArrayList	LinkedHashMap, List, Streams API
Error handling	Returned message strings	Custom LibraryException + try-catch
Search	Not implemented	Streams-based search by ID/title/author
Code style	Beginner friendly, procedural OOP	Maintainable, service-based design
Development time	Slower (all logic hand-written)	Faster (AI accelerated boilerplate and refactors)
Readability	Simple but verbose findBook/findMember loops	Cleaner intent via Streams, but requires familiarity with records

Development Time

Version A took less time to design conceptually because the pattern (classes + ArrayLists + linear search) is simple and familiar. Most of the time was spent writing repetitive lookup and menu handling code by hand. Version B took less overall time to reach a more feature complete state because AI assistance accelerated the introduction of records, Streams, persistence, and custom exceptions features that would otherwise require additional research and boilerplate.

Code Quality

Version B demonstrates higher code quality: immutability reduces the risk of unintended state mutation, validation is centralized in a required() helper instead of being duplicated, and persistence logic is isolated inside the service layer rather than mixed into the menu code.

Error Handling

Version A communicates failures as plain strings that the caller must remember to check and print. Version B uses a checked Library Exception, which forces calling code to handle errors explicitly and prevents an error from being silently ignored.

Readability and Maintainability

Version A is easier for a beginner to read line by line but becomes harder to extend, since every new rule adds another if statement and every entity change means editing getters/setters directly. Version B is slightly harder to read for someone unfamiliar with records and Streams, but is significantly easier to extend and maintain, since business rules are grouped inside LibraryService and data classes are self validating and immutable.

2.6 AI Prompts Used

1. Convert my Python Library Management System assignment into Java while keeping Version A and Version B.
2. Create a traditional Java OOP implementation using classes, ArrayLists, and simple menu logic.
3. Improve the Java implementation using records, Streams API, custom exceptions, and persistence.
4. Add comments beginning with "// AI Suggestion:" to identify AI-assisted improvements.

5. Generate PlantUML class, activity, and sequence diagrams for the Java Library Management System.

2.7 Manually Written vs AI-Generated Code Sections

Code Section	Origin
Book, Member, Library classes (Version A)	Manually written (traditional approach, no AI assistance)
Console menu loop (Version A)	Manually written
Book, Member records (Version B)	AI-assisted (converted from plain classes on request)
LibraryException custom exception	AI-suggested (marked with "// AI Suggestion:" comment)
searchBooks() Streams implementation	AI-suggested (marked with "// AI Suggestion:" comment)
save()/load() file persistence logic	AI-suggested (marked with "// AI Suggestion:" comment)
Console menu loop (Version B)	Manually adapted from Version A structure, reviewed after AI assistance
UML diagrams (Use Case, Class, Activity, Sequence)	AI-generated from the final class structure and prompts, reviewed and corrected manually

Task 3: Critical Analysis

3.1 How AI Changed the Coding Approach

Working with AI assistance shifted the development process from writing every line manually toward describing the desired behaviour and then reviewing, testing, and adjusting the generated result. For Version B, instead of manually deciding how to add persistence, the AI tool was asked to add file based storage, and the resulting code was then read carefully, traced through by hand, and tested to confirm it matched the intended behaviour of Version A. This changed the workflow from "write, then debug" to "specify, review, and refine", which required a different kind of attention: closely reading unfamiliar code rather than producing it from scratch.

3.2 Did AI Improve or Reduce Understanding?

AI assistance had a mixed effect on understanding. On one hand, it improved exposure to modern Java features records, the Streams API, and structured exception handling were adopted more quickly than they would have been through independent research alone, and comparing the AI suggested code to the original Version A deepened understanding of why these patterns are considered improvements. On the other hand, accepting AI suggested code without tracing through it line by line could easily reduce understanding, since it is possible to copy a working solution without internalising the reasoning behind it. Requiring every AI assisted section to be explicitly labelled ("// AI Suggestion:") and manually verified helped mitigate this risk in this assignment.

3.3 When AI Should NOT Be Used

- When the developer does not yet understand the underlying concept, since accepting AI output at that stage prevents genuine learning.
- For security critical code (authentication, payment processing, cryptography) without independent expert review, since subtle AI mistakes can introduce serious vulnerabilities.
- In academic settings where the assessment is specifically designed to test independent understanding, unless AI use is explicitly permitted and disclosed, as in this assignment.
- For final business logic decisions that carry legal, financial, or safety consequences, where a human domain expert must remain accountable for the outcome.

- When working with proprietary or sensitive code that should not be shared with a third-party AI service due to confidentiality obligations.

3.4 Ethical Considerations in AI Generated Code

- Attribution and transparency AI assisted sections should be clearly marked, as done in this project, so reviewers understand which code was human written and which was AI suggested.
- Accountability the developer, not the AI tool, remains responsible for verifying correctness, security, and fitness for purpose before code is shipped.
- Bias and training data provenance AI tools are trained on large public code corpora, and generated code can inadvertently mirror licensed or copyrighted patterns.
- Over trust professional looking, confidently written AI output can be mistaken for correct output, so independent testing remains essential.
- Data privacy sending proprietary code or sensitive data to a third party AI service can create confidentiality and compliance risks that must be managed by the organisation.

Task 4: Conclusion

This assignment demonstrated that AI assisted programming can meaningfully accelerate software development and improve code quality when used with careful review rather than blind acceptance. Building the same Library Management System twice once by hand and once with AI assistance made the practical trade offs concrete rather than theoretical: Version A was slower to extend but fully understood line by line, while Version B was faster to enrich with modern features but required disciplined verification of every AI suggested change.

In programming education, AI tools are most valuable as a way to see idiomatic examples of concepts a student is actively learning, provided the student still traces through and tests the generated code rather than treating it as a black box. In industry, the same principle applies at a larger scale: AI assisted programming is a genuine productivity tool for boilerplate, refactoring, and documentation, but human developers must retain ownership of design decisions, security review, and final accountability for shipped code. Used this way, AI-assisted programming is best understood as a capable collaborator that amplifies a developer's own skill, rather than a replacement for programming knowledge and judgement.

GitHub Repository

Source code, documentation, project files, and commit history for this assignment are available at:

[github.com/kavi-cs]